DESIGN DOCUMENT

TriggerTrace Bot

Find which application triggered a communication template

Document	Application design document
Version	1.0
Date	29 October 2026
Status	Draft for review
Prepared by	Amin Mir
Environments covered	Non-production (NP) and production (PR)

Contents

1 Executive summary

2 Business use case

3 Problem statement

4 Solution – how TriggerTrace resolves the problem

5 Architecture

6 Technology used

7 Security and operations

8 Limitations and next steps

A Glossary

1. Executive summary

TriggerTrace Bot is an internal web application that answers one question quickly: **which application triggered this communication template?**

A user types a plain sentence, for example *“Can you search triggering application of NOTIFY_DATA_ROLLOVER_KOODO for the last 5 days”*. TriggerTrace reads the sentence, searches the Communication API logs in Google Cloud Logging for the right environment, removes repeated records, finds the calling application in each log and shows the result in two parts:

- a short, plain-language **AI Summary** of what was searched and what was found; and
- a **Triggering Application** table next to each log with the consumer, request identifiers, notification type and sender, plus a remark when the request was internal.

Work that used to need Logs Explorer queries and careful reading of long request lines is reduced to one sentence and a few seconds. During testing, searches finished in roughly 2 to 18 seconds depending on the period and the amount of logging.

> **In one line:** ask in plain English, get the calling application with the evidence behind it, without writing a single log query.

2. Business use case

2.1 Background

Customers receive SMS and email notifications that are produced from named templates, such as `NOTIFY_DATA_ROLLOVER_KOODO` or `TELUS_WELCOME`. Applications ask the Communication API to send a template. Requests normally reach the Communication API through the Kong API gateway, which stamps each request with the calling consumer and with request identifiers. Every request is written to Google Cloud Logging, in a non-production (NP) project and a production (PR) project.

2.2 Business need

Teams that run, support or depend on notifications regularly need to know who is behind a template. The typical situations are:

Scenario	Typical user	Question to answer	Value
Unexpected or duplicate message	Support and operations	Which application sent this template, and how often?	Faster triage; the right team is contacted first time.
Change or retire a template	Template owners, product	Who still uses this template?	Avoids breaking live callers.
Ownership and audit	Governance, platform	Which consumer is registered against this template?	Clear ownership and evidence that can be shown to auditors.
Through the gateway or internal?	Platform and API team	Did this request come through Kong or from inside the platform?	The investigation starts in the right place.
Test versus live	QA and testers	Is the same template triggered the same way in NP and PR?	Safer releases.

The scenarios above describe the intended use of the tool; their relative priority should be confirmed with the business owners.

2.3 Users and stakeholders

- **Support and operations** – investigate customer-visible notification issues.
- **Template and application owners** – understand who depends on a template before changing it.
- **Platform and API team** – own the Communication API and the gateway; today they are the people most often asked.
- **QA and testers** – compare behaviour between environments.
- **Security and governance** – need access to production data to stay controlled and read-only.

2.4 Goals and success criteria

- An answer in seconds from a plain sentence, with no knowledge of the logging query language.
- One consistent answer instead of manual searches that depend on who is asking.
- The evidence is shown with the answer (consumer, identifiers, request count, first and last time seen), so anyone can verify it.

- Safe by design: read-only access, credentials hidden, secrets kept in a secret store.
- Both environments, NP and PR, available from the same page with one switch.

3. Problem statement

3.1 How the question is answered today

1. Open Google Cloud Logs Explorer and choose the right project, NP or PR.
2. Write a query with the template name and a time range, and work out which log source and bucket hold the Communication API logs.
3. Scroll through very large result sets and open entries one by one.
4. Read a long `request [{ ...20+ headers... },{ ...JSON body... }]` line to find the consumer name and the Kong request identifiers.
5. Repeat for the other environment. For logs that name no caller, search related logs by hand and make a judgement.
6. Map the consumer to an application and write the answer up manually.

3.2 Pain points

Problem	What it looks like in practice
Skill barrier	Needs the logging query language, knowledge of which project and log source to use, and access to the right log bucket.
Data volume	Observed during development: one Logs Explorer view held about 104,000 entries for just two days, and a single template produced 625 identical requests in one day. The useful answer is usually one record.
Hidden information	The caller is buried in a very long request line among many other headers and a large JSON body.
Inexact search	The built-in text search is not exact: searching `BYOD_ORDER_CONF_KOODO` also returns `BYOD_ORDER_CONF_KOODO_CSAG`, so results must be filtered by eye.
No caller in the log	Internal and pub/sub requests carry no gateway headers. The answer has to be pieced together from other logs.
Sensitive content	Request logs contain bearer tokens and customer data. Copying them into chats or tickets is a security risk.
Two environments	NP and PR are separate projects with separate permissions, so the work is repeated for each.
Person dependency	Only a few people know the platform well enough to do this quickly, so they are interrupted often.

3.3 Impact

- Slow triage of notification incidents and customer complaints.
- Repeated interruptions to platform engineers.
- Inconsistent answers, because each person searches in their own way.

- Risk of exposing credentials or customer data when log lines are copied around.

4. Solution – how TriggerTrace resolves the problem

4.1 Overview

TriggerTrace is a chat-style web page backed by a small API. The user asks in plain language. The system works out what to search for, reads the logs, collapses repeats, identifies the caller, and explains the result. Everything is read-only and nothing is stored by the application.

4.2 How each problem is resolved

Problem	What TriggerTrace does	Result
Skill barrier	A chat box takes a plain sentence. The AI gateway (FuelIX) picks out the template name, the environment and the period. If it is unavailable, a built-in pattern reads the sentence instead.	No query language needed.
Data volume	Identical requests (same template and same consumer) are collapsed into one record with a request count, such as “Total Requests 625×”, plus the first and last time seen.	625 records become 1, with the evidence kept.
Hidden information	The caller is extracted from each log and shown in a table beside it: consumer, notification type, consumer ID, Kong request IDs, appId, originator application and sender. The consumer shows whether it was found in the request or the response.	The answer is visible at a glance.
Inexact search	Results are filtered to whole template names only, and only request and pub/sub logs are shown.	No look-alike names in the results.
No caller in the log	A “Remarks” row states when a request was internal and not initiated by Kong. For logs without a caller, a deeper analysis reads the other logs of the same template over at least seven days and suggests the most likely application, shown in a “Comments” row.	An evidence-based suggestion instead of a dead end.
Sensitive content	Authorization headers, bearer tokens and keys are hidden before logs reach the page, and every backend log line and error message is scrubbed.	Safe to share screens and copy results.
Two environments	An NP / PR switch in the page chooses the project. Each environment uses its own service account.	One page for both.
Person dependency	The search is self-service and always runs the same way, with live progress shown.	Consistent answers; fewer interruptions.

4.3 What the user sees

1. Type a request, for example *"Triggering application of NOTIFY_DATA_ROLLOVER_KOODO from Jan-2026 to Oct 2026"*, and press Enter.
2. A live list of steps shows what is happening: reading the request, understanding it, searching the chosen environment, counting the results, asking the AI gateway, and finally "Done".
3. The **AI Summary** appears, typed out in plain sentences. Small buttons let the user copy it, hear it read aloud, or mark it good or bad.
4. Each unique log is shown as a card. On the left is the request body; on the right is the **Triggering Application** table, and a tag such as "Total Requests 286×" says how many identical requests were seen.

If no period is typed, the last 30 days are searched and shown as "Default". A period can be typed in many ways, for example *today, yesterday, last 5 days, past 2 weeks, last month, Jan-2026 to Oct 2026, or 2026-01-15 to 2026-02-03*.

4.4 Key capabilities

Capability	Description
Natural-language request	Template, period and environment are understood from one sentence.
Period understanding	Relative periods, month ranges and exact dates are read by code, not by AI, so the same words always give the same range.
Whole-name search	Only exact template names match; look-alike longer names are excluded.
Duplicate collapse	Same template and same consumer become one record with a request count and first and last time seen.
Caller table	Consumer (with request or response), notification type (for example SMS or Email), IDs, sender and a remark for internal requests.
Deep analysis	For logs with no caller, related logs are studied and the best candidate is suggested with a confidence level and the count behind it.
AI Summary	A first-person, plain-language account of the search, with copy, read-aloud and feedback buttons.
Single-entry lookup	Pasting a Cloud Logging entry ID shows exactly that entry, whatever kind of log it is.
Resilience	Every external call has a time limit; if the AI gateway is down, the built-in pattern is used and the page says so.

4.5 How AI is used, and how it is kept honest

AI task (through FuelIX)	Guardrail
Read the typed sentence	The template name it returns must literally appear in the sentence; the period is parsed by code; unknown environments are dropped.
Find caller keys in each log	Only request logs are sent, with bearer tokens removed first. Each value it returns is checked against the log text, and a plain pattern is the backup.
Choose the likely application	It may only choose from names that actually appear in the related logs, with their counts shown. It cannot invent a name. If it fails, a plain ranking is used.

5. Architecture

5.1 Architecture diagram

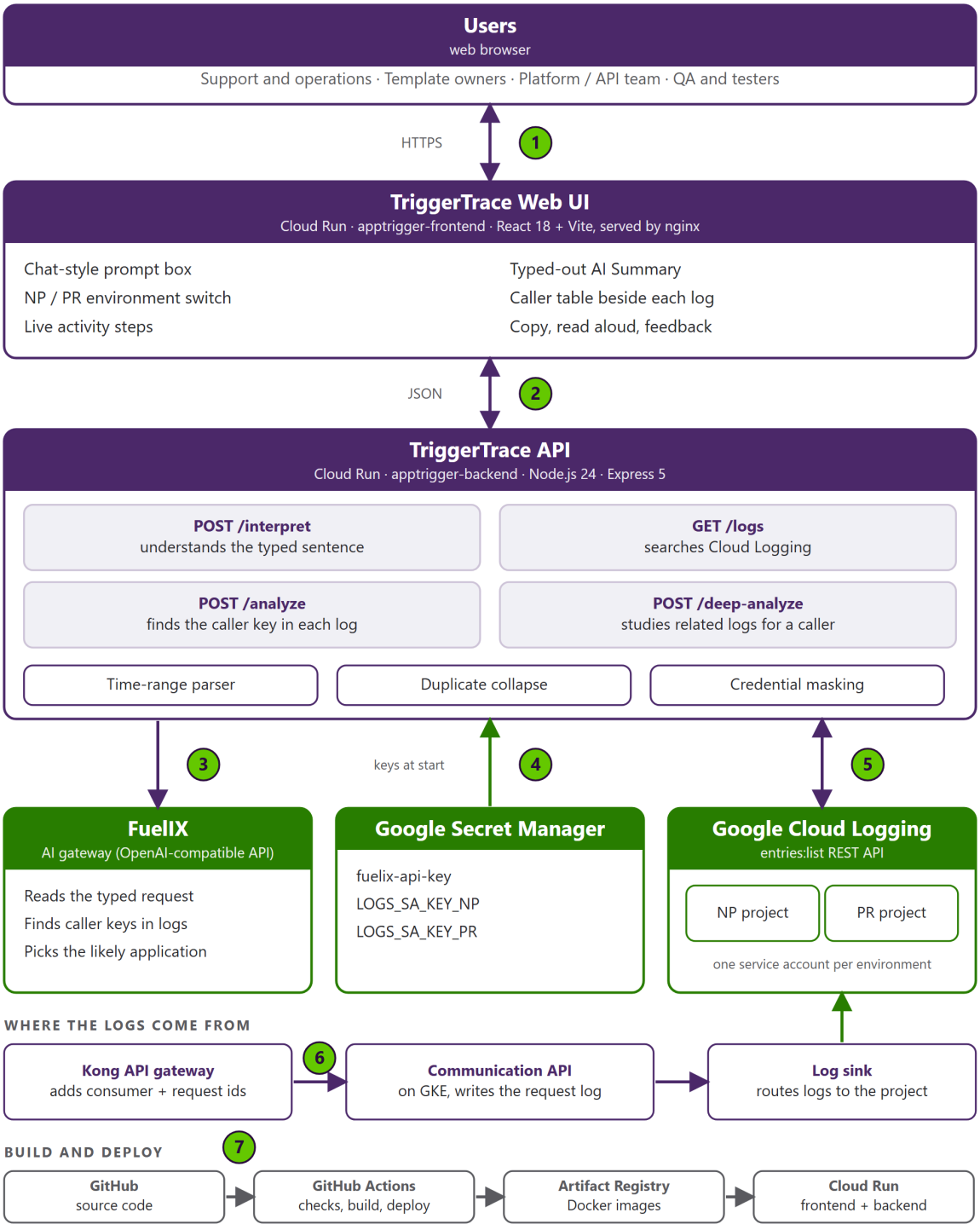


Figure 1. TriggerTrace architecture. Numbers refer to the steps in section 5.2.

TriggerTrace is two small services on Google Cloud Run, a web page and an API. The API reads logs from Cloud Logging, asks FuelIX for the AI steps, and gets its secrets from Secret Manager. The logs themselves are produced by the existing platform.

5.2 What happens at each numbered step

#	Step
1	The user opens the web page over HTTPS and types a request.
2	The page calls the API with JSON: `/interpret` (understand the sentence), `/logs` (search), `/analyze` (find the caller in each log) and `/deep-analyze` (study related logs). Each call has a time limit, so a stuck call ends with a clear message.
3	The API asks FuelIX to read the sentence, find caller keys and choose the likely application. Bearer tokens are removed first, and only answers that match the log text are kept.
4	When the service starts, Cloud Run gives the API its secrets from Secret Manager: the FuelIX key and one service-account key per environment.
5	The API reads the logs for the chosen environment through the Cloud Logging REST API, using that environment's own service account. Access is read-only.
6	Where the logs come from: callers reach the Communication API through the Kong gateway; the Communication API, running on GKE, writes the request log; a log sink routes it into the Cloud Logging project.
7	Build and deploy: code lives in GitHub. GitHub Actions run the checks, build the Docker images, push them to Artifact Registry and deploy both services to Cloud Run.

5.3 Components

Component	Responsibility	Where it runs
Web UI	Chat prompt, NP / PR switch, live steps, AI Summary, log cards, caller table, typed-out output.	Cloud Run (`apptrigger-frontend`, nginx)
API	Understands requests, searches logs, collapses repeats, hides credentials, calls FuelIX, runs the deep analysis, applies time limits and writes the operational log.	Cloud Run (`apptrigger-backend`, Node.js)
FuelIX	AI gateway used for the three AI tasks in section 4.5.	External service
Secret Manager	Holds the FuelIX key and the NP and PR service-account keys.	Google Cloud
Cloud Logging	Stores the Communication API logs for each environment.	NP and PR projects
Kong and Communication API	Produce the request logs that TriggerTrace reads. They are not changed by TriggerTrace.	Existing platform (GKE)
GitHub, Actions, Artifact Registry	Source control, checks, image build and deployment.	GitHub and Google Cloud

5.4 Environments

There are two environments. **NP** (non-production) and **PR** (production) are separate Google Cloud projects, each read by its own service account that holds only the Logs Viewer role. The NP / PR switch in the page chooses between them; the API refuses any other project. Nothing is written to either environment.

5.5 Data handling

- The application has **no database**. Logs are read at the moment of the search and are not stored.
- The browser keeps only small settings (for example the chosen environment) in its local storage.
- Credentials inside log entries are hidden before the entries leave the API.

5.6 Deployment

Item	Backend (API)	Frontend (web page)
Cloud Run service	`apptrigger-backend`	`apptrigger-frontend`
Image	Node.js 24 on Alpine Linux	Built with Node.js 24, served by nginx on Alpine
Size	512 MiB, 1 CPU, up to 5 instances, 300 s request limit	256 MiB, 1 CPU, up to 3 instances
Scaling	Scales to zero when idle; starts on demand.	
Access	Private by default. A deployment setting (`PUBLIC_ACCESS`) opens both services to anyone with the address and is logged as a warning when used.	
Secrets	FuelIX key and the NP / PR keys come from Secret Manager at start-up and are never stored in the image or printed.	

6. Technology used

Layer	Technology	Version	Purpose
Web UI	React	18.3	Single-page user interface.
	Vite	6.4	Build and development server.
	Plain CSS and web fonts (Poppins, Nunito)	–	TELUS-themed styling.
	Browser features	–	Clipboard (copy) and speech synthesis (read aloud).
	nginx (stable, Alpine)	–	Serves the built page on Cloud Run.
API	Node.js (long-term support)	24	Runtime for the backend.
	Express	5.2	HTTP routes.
	Google Cloud Logging client and Google Auth library	12.2 / 11.2	Sign-in and the Cloud Logging REST API.
	Built-in code modules	–	Period parsing, duplicate collapse, credential masking, deep analysis, operational logging.
AI	FuelIX gateway (OpenAI-compatible chat API)	–	Understands requests, finds callers, suggests applications. The model is a setting (an example is `claude-sonnet-4-5`).
Cloud	Google Cloud Run	–	Hosts both services.
	Google Cloud Logging	–	Source of the logs (NP and PR projects).
	Google Secret Manager	–	Stores keys.
	Google Artifact Registry	–	Stores Docker images.
	IAM service accounts	–	One reader account per environment (Logs Viewer).
	Google Kubernetes Engine (GKE)	–	Runs the Communication API that writes the logs.
Delivery	GitHub and GitHub Actions	–	Source control, automated checks, build and deployment.
	Docker (Alpine images)	–	Packaging for Cloud Run.
Quality	Automated check scripts	–	Cover syntax, period parsing, duplicate collapse, whole-name matching and credential masking; run on every change.

7. Security and operations

Area	How it is handled
Log access	Read-only. One service account per environment with the Logs Viewer role. The API accepts only the two configured projects.
Secrets	Stored in Secret Manager and handed to the service at start-up. Never in the image, never printed.
Credentials in logs	Authorization headers, bearer tokens, tokens and keys are hidden before log entries reach the page. Every backend log line and error message is scrubbed as a second line of defence.
AI data	Only request logs are sent to the AI gateway, with bearer tokens removed. AI answers are checked against the log text before use.
Network exposure	Both services are private by default. Public access is a deliberate setting and produces a warning.
Time limits	Google calls 30 s, FuelIX calls 60 s, a search stops reading after 45 s, and each page call is limited so nothing waits forever.
Observability	The API prints timed, labelled lines for every request, every page read from Cloud Logging and every AI call, and states at start-up which environments, key sources and AI settings are in use.
Data retention	The application stores no data.

8. Limitations and next steps

8.1 Known limitations

- **No sign-in in the application.** Protection currently relies on Cloud Run being private. A public deployment would expose production logs and AI usage.
- **Search limits.** Up to 500 unique records, 10 pages and about 45 seconds per search. Very large or very old searches can be partial, and repeat counts may then be low. The backend log says when this happens.
- **Callers exist only if the log carries them.** Internal requests have no gateway details; for these the page shows a remark and, where possible, a suggestion from related logs.
- **AI output is a suggestion.** The evidence (counts, identifiers, times) is always shown beside it.
- **Feedback buttons are not stored.** The good and bad buttons only mark the answer on the page.

8.2 Recommended next steps

#	Step	Why
1	Put Identity-Aware Proxy or single sign-on in front of the application.	Allows safe access from outside, with named users and an audit trail.
2	Use a custom, shorter address behind a load balancer.	Easier to remember and share; needed for step 1 in most designs.

3	Record feedback and search usage.	Measure how often answers are useful and improve the AI steps.
4	Move the deployment key to keyless sign-in (Workload Identity Federation).	Removes the last long-lived key from the build pipeline.
5	Move to the next Node.js long-term-support release when it is promoted.	Stay on a supported, patched runtime.
6	Add a team or application registry lookup.	Turn a consumer name into an owning application and team automatically.

Appendix A. Glossary

Term	Meaning
Template	A named message definition (for example `TELUS_WELCOME`) that produces an SMS or an email.
Consumer	The caller that the API gateway recorded for a request (the `X-Consumer-Username` value).
NP / PR	Non-production and production environments, each a separate Google Cloud project.
ECP	The platform whose requests are logged. A request into it is “To ECP” and its answer is “From ECP”; the page marks a consumer as found in the request or in the response.
Kong	The API gateway that adds the consumer and request identifiers to a request.
Internal request	A request that carries none of Kong’s identifiers, so it did not come in through the gateway.
FuelIX	The AI gateway used to understand requests and read logs.
Cloud Run	Google Cloud service that runs the two TriggerTrace services.
Cloud Logging	Google Cloud service where the platform’s logs are stored and searched.
Log entry ID (insertId)	The unique identifier of one Cloud Logging entry.
LTS	Long-term support: a Node.js release that receives fixes for an extended period.

Version 1.0 · 29 October 2026 · Draft for review